

Take-Home Assignment - Doc Q&A Portal (Next.js + AWS + Pinecone)

Goal: Build a small “Doc Q&A” app where:

- You can **upload plain-text documents**.
- You can **ask questions**.
- An **LLM** answers using those documents via **Pinecone** (RAG).

Target time: **~4–6 hours**. Don't overbuild. If you run out of time, explain trade-offs in the README.

Core Requirements

1) Frontend – Next.js + TypeScript

- Use **Next.js + TypeScript** (App or Pages Router).
- Two simple pages:
 - `/docs` (or similar):
 - Form to add one or more documents with `id`, `title`, `content` (plain text in a textarea).
 - Calls backend `POST /ingest` and shows success/error.
 - `/` (or `/ask`):
 - Input for a natural-language **question**.
 - On submit, calls backend `POST /ask`.
 - Displays `answer` and list of `sources` (doc id + title).

No auth or login needed.

2) Backend – API Gateway + Lambda (Node.js/TS)

Use **AWS API Gateway** → **Lambda** (Node.js, written in TypeScript).

Endpoints:

1. `POST /ingest`
 - Request body:

JSON

```
{
  "documents": [
    {
      "id": "refund-policy",
      "title": "Refund Policy",
      "content": "Full refund within 30 days with receipt. No refunds on digital goods."
    }
  ]
}
```

```
}  
]  
}
```

- Behavior:
 - **Chunk content** (any simple strategy; document it).
 - For each chunk:
 - Generate an **embedding**.
 - Upsert into **Pinecone** with:
 - **id** (e.g. `refund-policy#chunk-1`)
 - vector
 - metadata: at least `docId`, `title`, `chunkText`
 - Re-ingesting same **id** should update, not duplicate.
- Response example:

JSON

```
{  
  "ingestedDocuments": 1,  
  "ingestedChunks": 4  
}
```

2. POST /ask

- Request body:

JSON

```
{  
  "question": "Can I get a refund on a digital product?",  
  "topK": 3  
}
```

- Behavior:
 - Embed `question`.
 - Query **Pinecone** for `topK` similar chunks.
 - Build a prompt with **question + retrieved chunks**.
 - Call an **LLM** (OpenAI, Anthropic, etc.).
 - Return:

JSON

```
{
  "answer": "Digital products are not eligible for refunds.",
  "sources": [
    { "docId": "refund-policy", "title": "Refund Policy" }
  ]
}
```

- Handle:
 - No docs / no matches.
 - Pinecone/LLM errors.
 - Bad input.
-

Tech Constraints

- **Frontend:** Next.js + TypeScript.
- **Backend:** Node.js + TypeScript on AWS Lambda.
- **API:** AWS API Gateway → Lambda.
- **Vector store:** Pinecone (required).
- **LLM + embeddings:** Any hosted provider.
- Config via env vars (e.g. `PINECONE_API_KEY`, `PINECONE_INDEX`, `LLM_API_KEY`, etc.)

Infra-as-Code: Use **SAM**, **CDK**, or **Serverless Framework** to define:

- Lambdas for `/ingest` and `/ask`.
- API Gateway routes.
- IAM permissions.

Doesn't need to be perfect prod infra, just coherent.

Hard Rules

-  **No LangChain.**
-  No LlamaIndex or similar RAG frameworks.
-  You **can** use:
 - Pinecone SDK.
 - Official LLM/embedding SDKs.
 - `fetch` / `axios` / etc.

You should implement the RAG flow yourself:

`chunk → embed → store → query → build prompt → call LLM → answer.`

Optional (Nice-to-Have)

These are **bonus**, not required:

1. **SQS + async ingest**
 - Instead of doing all work inside `POST /ingest`, you can:
 - Write the doc to S3.
 - Publish a message to **SQS**.
 - Have a **second Lambda** subscribed to SQS that:
 - Reads doc from S3.
 - Does chunking + embeddings + Pinecone upserts.
 - If you do this, briefly explain the design in the README.
2. **File upload + text extraction**
 - PDFs/Docs + Azure Doc Intel / Textract / Tika, etc.
3. **Better RAG / UI polish**
 - Smarter chunking/ranking, nicer UX, etc.

If you don't have time, just describe what you'd add in the README.

What to Submit

- **Code:** GitHub repo or zip, including:
 - Next.js app.
 - Backend code + IaC.
 - **README:**
 - How to run locally.
 - Env vars needed.
 - High-level deploy steps.
 - Example requests for `/ingest` and `/ask`.
 - Assumptions, trade-offs, and "if I had more time, I would..."
 - **Tests:**
 - At least 1–2 small unit tests (e.g. chunking, RAG selection, prompt building).
-

How It'll Be Evaluated

- Code structure & separation of concerns.
- TypeScript usage (not a `any` swamp).
- How you implement RAG with Pinecone.
- How you wire API Gateway + Lambda.
- Clarity of README and overall reasoning.